

Guide de Déploiement Hound Search

Émetteur(s) : Connectiv-IT

Destinataire(s) : Connectiv-IT

Copie(s) :

Object : Guide de Déploiement : Hound Search sur Azure Stack (AlmaLinux)

CIT-Tools

Interne

Sopra Steria

Le 23 janvier 2026

Sommaire

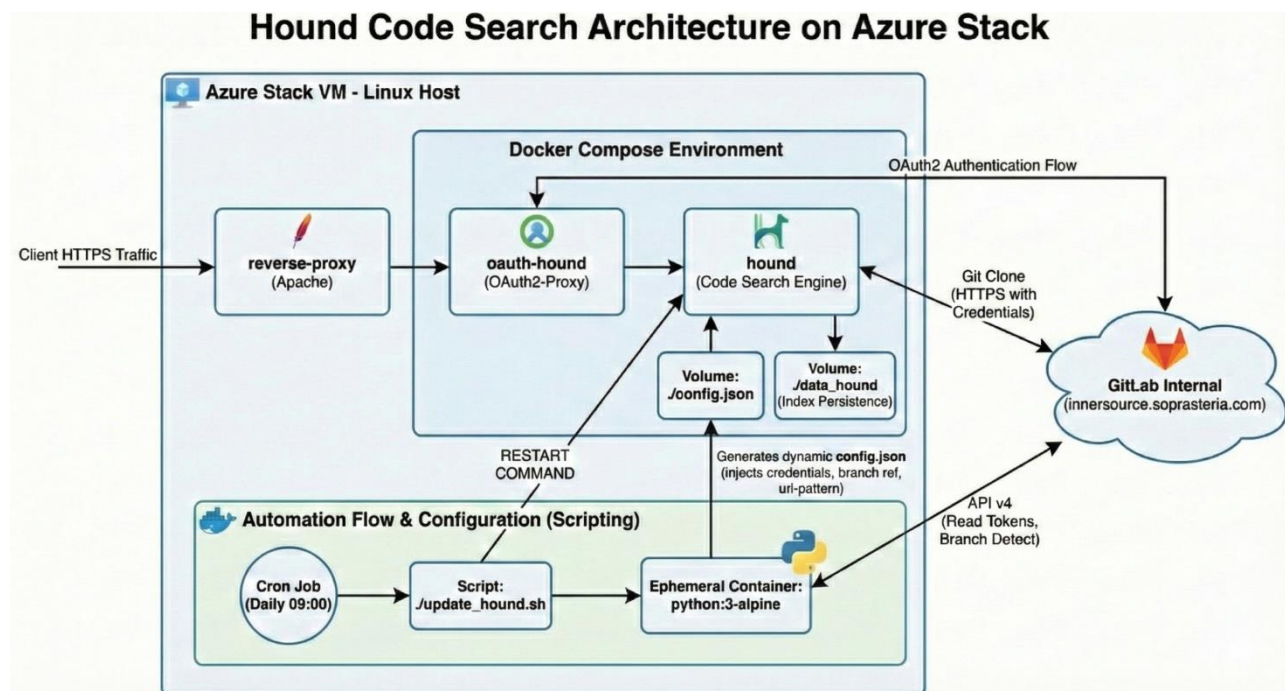
1. Introduction	3
1.1. Architecture	3
2. Préparation de l'Environnement	4
2.1. Installation Système (AlmaLinux)	4
2.2. Structure des Fichiers	5
2.3. Sécurité (OAuth & SSL)	5
2.4. Gestion des Secrets (.env)	5
2.5. Déploiement des services	7
3. Gestion de l'Indexation	9
3.1. Rôle (update_hound.sh)	9
3.2. Implémentation du Script	9
4. Maintenance et Mises à jour	10
4.1. Mise à jour Automatique AlmaLinux	10
4.2. Mise à jour Automatique des Conteneurs	10
4.3. Renouvellement des Certificats SSL	10

Le 23 janvier 2026

1. Introduction

Hound est un moteur de recherche de code extrêmement rapide, capable d'indexer des centaines de dépôts Git. Ce document décrit la mise en œuvre de Hound sur une VM Linux hébergée sur Azure Stack, sécurisée par HTTPS et une authentification OAuth2.

1.1. Architecture



La solution repose sur une architecture conteneurisée sur une VM Azure Stack (Linux Host), où Apache joue un rôle central de pivot :

- **OS** : AlmaLinux
- **Orchestrateur** : Docker.
- **Frontal** : Reverse-proxy (Apache).
- **Application** : Hound (Code Search Engine) et oauth-hound (OAuth2-Proxy).
- **Flux d'automatisation** : Un Cron Job journalier (09:00) lance le script /update_hound.sh pour la mise à jour et la configuration dynamique (config.json).
- **Persistance** : Volumes pour config.json et /data_hound (Index).

2. Préparation de l'Environnement

2.1. Installation Système (AlmaLinux)

Sur une VM fraîchement installée, nous devons configurer Docker. Comme nous sommes sur AlmaLinux, nous utilisons dnf

1. Mise à jour du système

```
sudo dnf update -y
```

2. Installation des outils de base (Éditeur de texte, Git, plugins DNF)

```
sudo dnf install -y nano git httpd mod_ssl dnf-plugins-core
```

3. Ajout du dépôt Docker officiel

```
sudo dnf config-manager --add-repo https://download.docker.com/linux/centos/docker-ce.repo
```

4. Installation de Docker et du plugin Compose

```
sudo dnf install -y docker-ce docker-ce-cli containerd.io docker-compose-plugin
```

5. Démarrage et activation au boot

```
Sudo systemctl enable docker --now  
Sudo systemctl enable --now httpd
```

6. Ajout de l'utilisateur courant au groupe docker

```
sudo usermod -aG docker $USER
```

7. Autoriser Apache à initier des connexions réseau

```
sudo setsebool -P httpd_can_network_connect 1
```

2.2. Structure des Fichiers

Dossier Application : /home/azureuser/cit-tools/

```
/home/azureuser/cit-tools/  
├── .env                # [IMPORTANT] Fichier de secrets (GitLab ID/Secret/Token)  
├── docker-compose.yml  # Orchestration des conteneurs  
├── config.json         # Configuration générée des dépôts Hound  
├── update_hound.sh     # Script de mise à jour quotidienne (Cron)  
├── update_containers.sh # Script de maintenance hebdomadaire  
└── data_hound/        # Dossier de persistance de l'index (Volume Docker)
```

Dossiers Système :

```
/var/certs/            # Stockage des certificats SSL (Permissions restreintes)  
├── server.crt  
└── server.key  
  
/etc/httpd/conf.d/     # Configuration Apache  
└── search-hound.conf
```

2.3. Sécurité (OAuth & SSL)

Dans GitLab, créez une Application OAuth (*Settings > Applications*) pour permettre l'authentification :

- **Name** : Hound Search Production
- **Redirect URI** : `https://slazcittools01.azy.azurestack.ssg/oauth2/callback`
- **Scopes** : `openid, profile, email`
- **Confidential** : Oui

Une fois l'arborescence en place, suivez cette procédure pour générer les fichiers dans le dossier `/var/certs/`.

Étape A : CSR avec OpenSSL

Exécutez ceci sur la VM pour générer la clé privée (`server.key`) et la demande (`certreq.pem`).

```
mkdir /var/certs
cd /var/certs
openssl req -newkey rsa:2048 -keyout server.key -out certreq.pem \
-subj "/C=FR/O=Sopra Steria/CN=cit-hound.azy.azurestack.ssg/emailAddress=contact-
equipe@soprasteria.com" \
-addext "subjectAltName = DNS:cit-hound.azy.azurestack.ssg,DNS:slazcittools01.azy.az-
urestack.ssg" \
```

-nodes

Champ	Description
<code>/CN=...</code>	Common Name. C'est le nom de domaine complet (FQDN) de votre VM Azure. Si ce nom change, le certificat devient invalide.
<code>emailAddress=...</code>	Contact. L'adresse email de l'équipe (ou la DL) qui recevra les notifications d'expiration de la PKI.
<code>subjectAltName</code>	SAN. Répétez le nom DNS ici. C'est obligatoire pour que les navigateurs modernes (Chrome/Edge) acceptent le certificat.

Étape B : Enrôlement et récupération

1. Copiez le contenu de `certreq.pem` sur le portail PKI (<https://pki1.is.soprasteria.com/certsrv>).
2. Utilisez le template : **SSG - SSL/TLS Other Server Authentication (Manual Renew)**.
3. Téléchargez le certificat (Base64), renommez-le en `server.crt`.
4. Placez `server.crt` à côté de `server.key` dans votre dossier `var/certs/`

2.4. Gestion des Secrets (.env)

Aucun secret ne doit apparaître en clair. Créez un fichier `.env` à la racine du projet.

Fichier : `/home/azureuser/cit-tools/.env`

```
GITLAB_CLIENT_ID=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
GITLAB_CLIENT_SECRET=xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
OAUTH_COOKIE_SECRET= GITLAB_PAT_TOKEN=glpat-xxxxxxxxxxxxxxxxxxxxxx
/home/azureuser/cit-tools/.env
```

2.5. Déploiement des services

Créez le fichier `/etc/httpd/conf.d/search-hound.conf`. **Configuration à modifier :** Remplacez `slazcitttools01.azy.azurestack.ssg` par le FQDN réel de votre VM si différent.

```
Listen 443

<VirtualHost *:80>
    ServerName cit-hound.azy.azurestack.ssg
    Redirect permanent / https://cit-hound.azy.azurestack.ssg/
</VirtualHost>

<VirtualHost *:443>
    ServerName cit-hound.azy.azurestack.ssg

    SSLEngine on
    SSLCertificateFile "/usr/local/apache2/conf/certs/server.crt"
    SSLCertificateKeyFile "/usr/local/apache2/conf/certs/server.key"

    ProxyPreserveHost On
    RequestHeader set X-Forwarded-Proto "https"
    RequestHeader set X-Forwarded-Port "443"

    ProxyPass "/" "http://oauth-hound:4180/"
    ProxyPassReverse "/" "http://oauth-hound:4180/"
</VirtualHost>
```

Fichier Apache : `search-hound.conf`

Directive	Description
<code>ServerName</code>	Le nom DNS public de votre VM. Doit être identique au CN du certificat.
<code>ProxyPass</code>	L'adresse de destination interne. Ici, nous pointons vers <code>oauth-hound</code> (nom du service Docker) sur le port 4180.
<code>SSLCertificate</code>	Chemins internes au conteneur. Ne modifiez pas ces chemins sauf si vous changez le <code>docker-compose.yml</code>

Créez le fichier `docker-compose.yml` à la racine. **Configuration à modifier** : Insérez vos Credentials GitLab et générez le Cookie Secret.

```
services:
  hound:
    build:
      context: https://github.com/hound-search/hound.git
      image: hound-search/hound
      container_name: hound-search
      command: -conf /hound/config.json -addr :6080
    volumes:
      - ./config.json:/hound/config.json
      - ./data_hound:/data
    restart: unless-stopped

  oauth-hound:
    image: quay.io/oauth2-proxy/oauth2-proxy:latest
    container_name: oauth-hound
    ports:
      - "127.0.0.1:4180:4180"
    env_file:
      - .env
    environment:
      OAUTH2_PROXY_PROVIDER: "gitlab"
      OAUTH2_PROXY_CLIENT_ID: "${GITLAB_CLIENT_ID}"
      OAUTH2_PROXY_CLIENT_SECRET: "${GITLAB_CLIENT_SECRET}"
      OAUTH2_PROXY_COOKIE_SECRET: "${OAUTH_COOKIE_SECRET}"
      OAUTH2_PROXY_OIDC_ISSUER_URL: "https://innersource.soprasteria.com"
      OAUTH2_PROXY_UPSTREAMS: "http://hound:6080/"
      OAUTH2_PROXY_HTTP_ADDRESS: "0.0.0.0:4180"
      OAUTH2_PROXY_REDIRECT_URL: "https://cit-hound.azy.azurestack.ssg/oauth2/callback"
      OAUTH2_PROXY_EMAIL_DOMAINS: "*"
      OAUTH2_PROXY_COOKIE_SECURE: "true"
      OAUTH2_PROXY_BANNER: "Hound Search"
    restart: unless-stopped
```

Fichier Orchestration : `docker-compose.yml`

3. Gestion de l'Indexation

3.1. Rôle (update_hound.sh)

Hound Search a besoin d'un fichier `config.json` pour savoir quels projets indexer. Ce script automatise ce processus en :

1. Interrogeant l'API GitLab pour lister vos projets.
2. Générant le JSON avec injection sécurisée du token.
3. Redémarrant le conteneur pour appliquer les changements

3.2. Implémentation du Script

Implémentation du script Créez le fichier `update_hound.sh`.

```
#!/bin/bash #
WORKDIR="/home/azureuser/cit-tools"
if [ -f "$WORKDIR/.env" ]; then
    source "$WORKDIR/.env"
else
    echo "Erreur : Fichier .env introuvable."
    exit 1
fi
GITLAB_URL="https://innersource.soprasteria.com"
echo "Début de la synchronisation..."
# 2. Utilisation de Python via Docker pour générer le JSON
docker run -i --rm \
    -v "$WORKDIR:/app" \
    -e GITLAB_TOKEN="$GITLAB_PAT_TOKEN" \
    -e GITLAB_URL="$GITLAB_URL" \
    python:3-alpine python - <<'EOF' import urllib.request, json, os, sys
# ... (Logique Python standard pour récupérer les repos GitLab) ...
EOF
if [ $? -eq 0 ]; then
    echo "Redémarrage de Hound..."
    cd $WORKDIR
    docker compose restart hound
else
    echo "Échec."
    exit 1
fi
```

N'oubliez pas : `chmod +x update_hound.sh`

Ajouter au Cron (`crontab -e`) :

```
0 9 * * * /home/user/hound-search/update_hound.sh >> /home/user/hound-search/up-
date_hound.log 2>&1
```

4. Maintenance et Mises à jour

4.1. Mise à jour Automatique AlmaLinux

Pour activer les mises à jour de sécurité automatiques :

1. Installation : `sudo dnf install -y dnf-automatic`
2. Configuration : Éditer `/etc/dnf/automatic.conf` et mettre `apply_updates = yes`.
3. Activation : `sudo systemctl enable --now dnf-automatic.timer`

4.2. Mise à jour Automatique des Conteneurs

Script `update_containers.sh` :

```
#!/bin/bash
cd /home/user/hound-search/
docker compose pull
docker compose up -d --remove-orphans
docker image prune -f
```

Rendre exécutable : `chmod +x update_containers.sh`

Ajouter au Cron (`crontab -e`) :

```
0 9 * * 1 /home/user/hound-search/update_containers.sh >> /home/user/hound-search/up-
date_containers.log 2>&1
```

4.3. Renouvellement des Certificats SSL

1. **Génération d'une nouvelle CSR** : Exécutez la même commande OpenSSL que lors de l'installation. Cela générera un nouveau fichier `certreq.pem` et une nouvelle clé `server.key`.
2. **Demande PKI**:
 - Connectez-vous au portail PKI (<https://pki1.is.soprasteria.com/certsrv>).
 - Soumettez le contenu du nouveau `certreq.pem`.
 - Téléchargez le nouveau certificat et renommez-le en `server.crt`.
3. **Remplacement** : Placez le nouveau `server.crt` dans le dossier `/var/certs`